



Lab#6 - Nested Loops, DIV and MUL

Nested loops using PUSH & POP

In x86 assembly programming, nested loops are similar in concept to those in high-level languages, where one loop is placed inside another. This allows for more complex iteration patterns, such as processing two-dimensional arrays or performing operations across multiple levels of data. Nested loops are implemented using the LOOP instruction, which decrements a register (typically ECX) and jumps back to a specified label until the register reaches zero. In a nested loop, each loop requires its own counter register, such as ECX for the outer loop and EBX for the inner loop.

To manage multiple loops and preserve counter values, stack manipulation instructions like PUSH and POP are often used to save and restore registers. This allows the inner loop to modify registers without interfering with the outer loop.

Instruction	Description	Example
PUSH	Pushes the value of a register or memory location onto the stack.	PUSH EAX ; Saves the value of EAX on the stack.
POP	Pops the top value from the stack into a register or memory location.	POP EBX ; Restores the value from the top of the stack into EBX.
PUSHAD	Pushes all general-purpose registers onto the stack (EAX, ECX, EDX, etc.).	PUSHAD ; Saves all general-purpose registers.
POPAD	Pops values from the stack into all general-purpose registers.	POPAD ; Restores all general-purpose registers from the stack.

Example:

```
INCLUDE Irvine32.inc
; Program to sum the elements of the array n number of times
.data
    array DWORD 10, 20, 30, 40, 50 ; A 1D array with 5 elements
    arraySize DWORD 5 ; Number of elements in the array
    repeatCount DWORD 3 ; Number of times to repeat the sum
    totalSum DWORD ? ; To store the total sum for all repetitions

.code
main PROC
    MOV EAX, 0 ; Clear EAX for the total sum
    MOV ECX, repeatCount ; Outer loop counter (number of repetitions)

outerLoop:
    PUSH ECX ; Save the outer loop counter (repeat count)

    ; Inner loop to sum the elements of the array
```

```

MOV ECX, arraySize ; Inner loop counter (array size)
MOV EBX, 0 ; Index for array element access
MOV ESI, 0 ; ESI will hold the sum of the current array iteration

innerLoop:
ADD ESI, array[EBX] ; Add the current element to ESI (current sum)
ADD EBX, 4 ; Increment the index
LOOP innerLoop ; Repeat until all array elements are summed

ADD EAX, ESI ; Add the sum of this repetition to the total sum

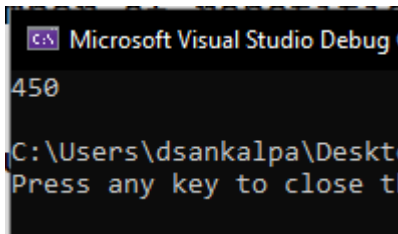
POP ECX ; Restore the outer loop counter
LOOP outerLoop ; Repeat the outer loop

MOV totalSum, EAX ; Store the total sum in totalSum

; Display the total sum
MOV EAX, totalSum
CALL WriteDec ; Output the result as a decimal number
CALL CrLf ; Move to the next line

EXIT
main ENDP
END main

```

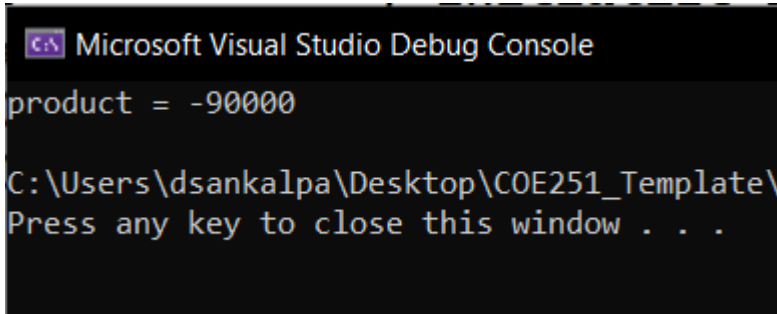


More arithmetic instructions

Instruction	Description	Example	Possible Operand Combinations
MUL	Unsigned multiplication. Multiplies the value in a register or memory by AX (16-bit) or EAX (32-bit). The result is stored in DX:AX (for 16-bit) or EDX:EAX (for 32-bit).	MUL BX ; Unsigned multiply AX by BX, result in DX:AX.	MUL reg MUL mem
IMUL	Signed multiplication. Multiplies the value in a register, memory, or an immediate by AX (16-bit) or EAX (32-bit). The result is stored in DX:AX (for 16-bit) or EDX:EAX (for 32-bit).	IMUL BX ; Signed multiply AX by BX, result in DX:AX.	IMUL reg IMUL mem IMUL reg, imm IMUL reg, reg, imm IMUL reg, mem, imm
DIV	Unsigned division. Divides DX:AX (16-bit) or EDX:EAX (32-bit) by the specified operand. The quotient is stored in AX (16-bit) or EAX (32-bit), and the remainder is stored in DX (16-bit) or EDX (32-bit).	DIV BX ; Unsigned division of DX:AX by BX, quotient in AX, remainder in DX.	DIV reg DIV mem
IDIV	Signed division. Divides DX:AX (16-bit) or EDX:EAX (32-bit) by the specified operand. The quotient is stored in AX (16-bit) or EAX (32-bit), and the remainder is stored in DX (16-bit) or EDX (32-bit).	IDIV BX ; Signed division of DX:AX by BX, quotient in AX, remainder in DX.	IDIV reg IDIV mem

Lab Work (Questions 1-4):

1. Given a BYTE array: myArray BYTE 10, 20, -30, 15, write an assembly program that calculates the product of all elements in the array. Display the final product to the user.



The screenshot shows the Microsoft Visual Studio Debug Console with a black background and white text. The first line displays 'product = -90000'. The second line shows the file path 'C:\Users\dsankalpa\Desktop\COE251_Template\' followed by 'Press any key to close this window . . .'.

Solution:

```
TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data
    myArray byte 10,20,-30,15
    str1 byte "product = ", 0

.code
main PROC
    mov eax, 1

    mov edx, offset str1
    call writestring

    mov esi, offset myArray

    mov ecx, lengthof myArray

    again:
    movsx ebx, byte ptr [esi]
    imul eax, ebx
    inc esi
    loop again

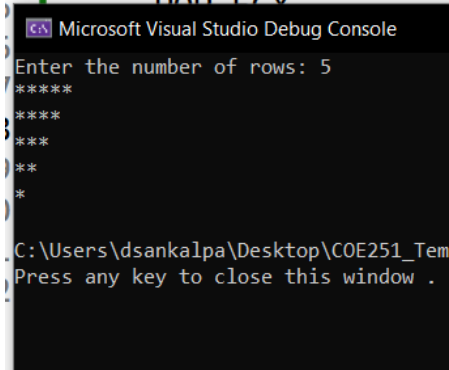
    call writeint
    call crlf

    exit
main ENDP
END main
```

Screenshot:

```
product = -90000
```

2. Write an assembly program that prompts the user to enter a number n. The program should then display a right-angled triangle pattern of asterisks *, with n rows.



```
Microsoft Visual Studio Debug Console
Enter the number of rows: 5
*****
*****
****
***
**
*
```

C:\Users\dsankalpa\Desktop\C0E251_Tem
Press any key to close this window .

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

str1 byte "Enter the number of rows: ", 0

str2 byte "*", 0

.code

main PROC

mov edx, offset str1

call writestring

call readint

mov edx, offset str2

mov ecx, eax

outer:

push ecx

mov ecx, eax

inner:

call writestring

loop inner

dec eax

```

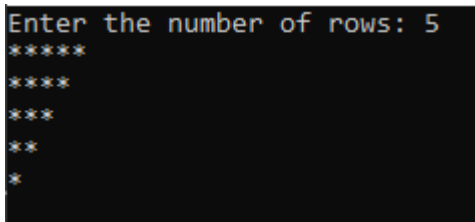
        call crlf
        pop ecx

        loop outer

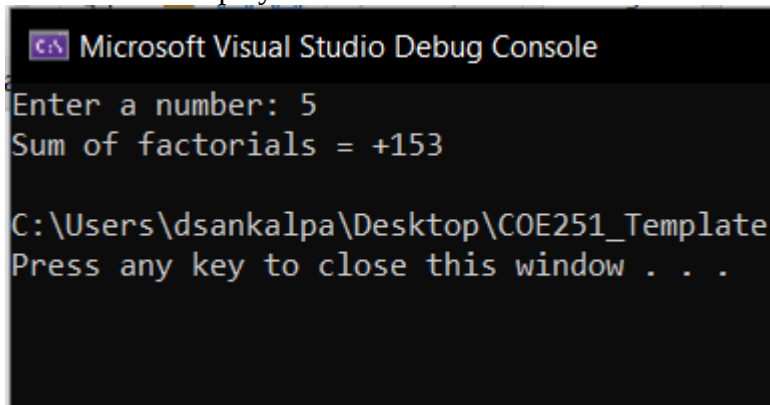
        exit
    main ENDP
END main

```

Screenshot:



- Write an assembly program that calculates the sum of factorials of numbers from 1 to n, where n is a number entered by the user. For example, if the user enters n = 4, the result should be $1! + 2! + 3! + 4! = 33$. Display the result to the user.



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```

str1 byte "Enter a number: ", 0
str2 byte "Sum of factorials = ", 0
sum dword 0

```

.code

main PROC

```

    mov edx, offset str1
    call writestring
    call readint

```

```

    mov edx, offset str2

```

```

        call writestring

        mov ecx, eax
        mov eax, 0

outer:
    push ecx

    mov eax, 1

inner:
    mul ecx
    loop inner

    add sum, eax

    pop ecx

    loop outer

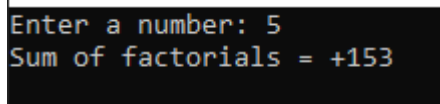
    mov eax, sum
    call writeint

    call crlf

    exit
main ENDP
END main

```

Screenshot:



```

Enter a number: 5
Sum of factorials = +153

```

4. Write an assembly program that reads 5 integers from the user. The integers can be both positive and negative. Calculate the average of the 5 numbers and display both the **quotient** and **remainder** to the user.

```
Microsoft Visual Studio Debug Console

Enter a number: 1234
Enter a number: -1000
Enter a number: -250
Enter a number: 500
Enter a number: 120
Quotient = +120
Remainder = +4

C:\Users\dsankalpa\Desktop\COE251_
Press any key to close this window
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter a number: ", 0
str2 byte "Quotient = ", 0
str3 byte "Remainder = ", 0
sum dword 0
count dword 5
```

.code

main PROC

```
mov ecx, count
```

```
again:
```

```
mov edx, offset str1
```

```
call writestring
```

```
call readint
```

```
add sum, eax
```

```
loop again
```

```
mov eax, sum
```

```
mov ebx, count
```

```
CDQ
```

```
idiv ebx
```

```
mov ebx, edx
```

```
mov edx, offset str2
```

```
call writestring
```

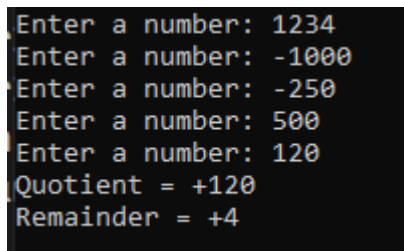
```
call writeint
```

```
    mov eax, ebx
    call crlf
    mov edx, offset str3
    call writestring
    call writeint

    call crlf

    exit
main ENDP
END main
```

Screenshot:



```
Enter a number: 1234
Enter a number: -1000
Enter a number: -250
Enter a number: 500
Enter a number: 120
Quotient = +120
Remainder = +4
```